

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO  
PAULO  
CAMPUS SÃO PAULO**

**BACHARELADO EM SISTEMAS DE INFORMAÇÃO**

**ANDREY GUSTAVO SEGANTIN DOS SANTOS  
ELTON LIMA ARAUJO  
JOÃO GUSTAVO DOS SANTOS**

**SISTEMA DE GERENCIAMENTO DE INSTITUIÇÃO FINANCEIRA**

**SÃO PAULO  
2025**

**ANDREY GUSTAVO SEGANTIN DOS SANTOS**

**ELTON LIMA ARAUJO**

**JOÃO GUSTAVO DOS SANTOS**

**BACHARELADO EM SISTEMAS DE INFORMAÇÃO**

**SISTEMA DE GERENCIAMENTO DE INSTITUIÇÃO FINANCEIRA**

Trabalho apresentado ao Curso de Bacharelado em Sistemas de Informação, do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Campus de São Paulo, como requisito parcial para aprovação na disciplina Banco de Dados 2, sob orientação do Professor Doutor Francisco Verissimo Luciano.

**SÃO PAULO**

**2025**

**Resumo**

O presente trabalho descreve o desenvolvimento de um sistema bancário digital simplificado, simulando as operações essenciais de uma fintech, denominada Instituição Financeira de São Paulo (IFSP). O objetivo principal foi projetar e implementar uma solução completa de software, integrando um banco de dados relacional robusto a uma aplicação web interativa. A metodologia adotada envolveu a construção de uma arquitetura cliente-servidor baseada em API REST. O back-end foi desenvolvido na linguagem Python, utilizando o microframework Flask e a biblioteca psycopg2 para manipulação direta e otimizada do banco de dados PostgreSQL, hospedado na plataforma de nuvem Render. O front-end foi construído utilizando HTML5, CSS3 e o framework Bootstrap, operando localmente e comunicando-se com o servidor através de requisições HTTP assíncronas. O sistema resultante permite a gestão de clientes e contas, além da execução segura de transações financeiras como depósitos, saques e transferências, garantindo a integridade dos dados e validando o uso de tecnologias open source no desenvolvimento de soluções financeiras escaláveis.

**Palavras-chaves:** Sistema Bancário. API REST. Python Flask. PostgreSQL. Computação em Nuvem.

## SUMÁRIO

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| 1. INTRODUÇÃO .....                                                           | 4  |
| 2. SOLUÇÃO PROPOSTA .....                                                     | 5  |
| 3. OBJETIVOS .....                                                            | 6  |
| 3.1. Objetivo Geral.....                                                      | 6  |
| 3.2. Objetivos Específicos .....                                              | 6  |
| 4. TECNOLOGIAS .....                                                          | 7  |
| 4.1 Back-end .....                                                            | 7  |
| 4.2. Banco de Dados.....                                                      | 7  |
| 4.3. Controle de Versão.....                                                  | 7  |
| 4.4. Front-end .....                                                          | 7  |
| 4.5. Infraestrutura .....                                                     | 7  |
| 5. CRONOGRAMA DE ATIVIDADES .....                                             | 8  |
| 5.1. Fase 1: Análise e Projeto (Semanas 1 a 2) .....                          | 8  |
| 5.2. Fase 2: Desenvolvimento do Back-end (Semanas 3 a 5) .....                | 8  |
| 5.3. Fase 3: Desenvolvimento do Front-end e Integração (Semanas 6 a 8) .....  | 9  |
| 5.4. Fase 4: Testes Finais, Implantação e Documentação (Semanas 9 a 10) ..... | 9  |
| 6. DEFINIÇÕES DE REQUISITOS .....                                             | 9  |
| 6.1. Requisitos Funcionais (RF).....                                          | 10 |
| 6.2. Requisitos Não Funcionais (RNF) .....                                    | 11 |
| 7. PROJETO DE SOFTWARE .....                                                  | 12 |
| 7.1. Casos de Uso .....                                                       | 12 |
| 7.2. Diagrama de Atividades .....                                             | 14 |
| 7.3. Diagrama de Classes .....                                                | 16 |
| 8. BANCO DE DADOS .....                                                       | 18 |
| 8.1. Projeto Conceitual .....                                                 | 18 |
| 8.2. Projeto Lógico .....                                                     | 19 |
| 8.3. Projeto Físico .....                                                     | 21 |
| 8.3.1. Definições de Estrutura (DDL) .....                                    | 22 |
| 8.3.2. Lógica de Negócio no Banco (Stored Procedures).....                    | 22 |
| 8.3.3. Auditoria e Segurança (Triggers) .....                                 | 23 |
| 8.3.4. Camada de Apresentação (Views).....                                    | 23 |
| 9. PROPOSTA COMERCIAL .....                                                   | 23 |
| 9.1. Investimento em Desenvolvimento (CAPEX) .....                            | 24 |

|                                                        |    |
|--------------------------------------------------------|----|
| 9.2. Custos Operacionais e Infraestrutura (OPEX) ..... | 24 |
| 9.3. Retorno sobre o Investimento (ROI).....           | 25 |
| 10. CONSIDERAÇÕES FINAIS .....                         | 25 |
| 11. REFERÊNCIAS BIBLIOGRÁFICAS .....                   | 26 |

## **1. INTRODUÇÃO**

A progressiva digitalização dos serviços financeiros, catalisada pelo surgimento das tecnologias financeiras (Fintechs), estabelece novos paradigmas para a arquitetura de sistemas de informação. Este fenômeno não se limita à automação de tarefas, mas representa uma reconfiguração profunda dos modelos de negócio e da oferta de valor no setor, impulsionando a inovação em um ritmo sem precedentes (PUSCHMANN, 2017). A transição das operações bancárias para plataformas digitais demanda, consequentemente, soluções de software que não apenas assegurem a eficiência transacional, mas que também garantam elevados níveis de segurança, integridade e escalabilidade.

Neste cenário, a correta modelagem da camada de persistência de dados e da lógica de negócio do servidor emerge como um componente crítico para a sustentabilidade e confiabilidade das operações financeiras. O presente trabalho tem como propósito documentar o processo de desenvolvimento de um protótipo de software que simula as operações de uma instituição de pagamentos. O projeto adota um processo de desenvolvimento estruturado, que, conforme preconizado pelas boas práticas da engenharia de software, abrange as fases de levantamento e análise de requisitos, projeto de software, implementação e testes (PRESSMAN; MAXIM, 2019). A solução tecnológica emprega Python, com o microframework Flask, para a construção do back-end; HTML, CSS e JavaScript, com o auxílio do framework Bootstrap, para a interface do usuário; e o SGBD PostgreSQL para a gestão dos dados.

Dessa forma, este estudo visa apresentar uma aplicação prática dos princípios de engenharia de software e modelagem de banco de dados na construção de um sistema transacional, servindo como um estudo de caso para o desenvolvimento de plataformas análogas no domínio financeiro.

## **2. SOLUÇÃO PROPOSTA**

Para atender aos objetivos delineados, propõe-se o desenvolvimento de um sistema web completo que simula as operações essenciais de uma instituição de pagamentos. A solução será construída seguindo uma arquitetura de três camadas (three-tier architecture), um padrão consolidado que promove a separação de responsabilidades e a manutenibilidade do sistema.

Esta arquitetura desacopla a aplicação nas seguintes camadas lógicas:

- Camada de Apresentação (Frontend): Responsável pela interface com o usuário (UI) e pela experiência do usuário (UX). É nesta camada que os dados são exibidos e as interações são capturadas.
- Camada de Aplicação/Negócio (Backend): Onde reside a lógica de negócio do sistema. Esta camada processa as requisições da camada de apresentação, aplica as regras de negócio (como validações de saldo, autorização de transações, etc.) e coordena a interação com a camada de dados.
- Camada de Dados (Banco de Dados): Encarregada da persistência e do gerenciamento dos dados. Garante que as informações sejam armazenadas de forma segura, íntegra e consistente.

### **3. OBJETIVOS**

#### **3.1. Objetivo Geral**

O objetivo geral deste trabalho é projetar, desenvolver e documentar um protótipo funcional de uma aplicação web para simulação de pagamentos digitais, aplicando as fases fundamentais do processo de desenvolvimento de software e utilizando tecnologias modernas para a sua implementação.

#### **3.2. Objetivos Específicos**

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Especificar e desenvolver os módulos de software responsáveis pelo gerenciamento do ciclo de vida de clientes e suas respectivas contas digitais, contemplando as operações de cadastro, autenticação e consulta de informações.
- Projetar e construir a camada de serviço (back-end) em Python, utilizando o microframework Flask, para implementar a lógica de negócio e expor os recursos do sistema por meio de uma API.
- Elaborar a camada de apresentação (front-end) de forma intuitiva e responsiva, utilizando HTML, CSS, JavaScript e Bootstrap, para permitir a interação do usuário com o sistema.
- Estruturar o modelo de dados relacional e implementá-lo no SGBD PostgreSQL, visando garantir a integridade, consistência e persistência dos dados da aplicação.

- Desenvolver e validar os mecanismos transacionais para operações financeiras essenciais, como depósitos, saques e transferências, assegurando a confiabilidade de cada operação.

## **4. TECNOLOGIAS**

Para o desenvolvimento deste projeto, foi selecionado um conjunto de tecnologias modernas e robustas, capazes de atender aos requisitos de segurança, desempenho e escalabilidade de uma aplicação financeira. A seguir, detalham-se as ferramentas e plataformas escolhidas para cada componente da arquitetura do sistema.

### **4.1 Back-end**

O servidor da aplicação foi desenvolvido em Python utilizando o microframework Flask. A escolha deve-se à leveza e simplicidade do Flask para a construção da API. A comunicação com o banco de dados é realizada através da biblioteca psycopg2, um driver PostgreSQL para Python que permite a execução de comandos SQL de forma eficiente e segura, garantindo alto desempenho na manipulação dos dados.

### **4.2. Banco de Dados**

O Sistema Gerenciador de Banco de Dados (SGBD) utilizado é o PostgreSQL. A escolha se justifica pela sua robustez, confiabilidade e estrita conformidade com o padrão ACID (Atomicidade, Consistência, Isolamento e Durabilidade), características essenciais para garantir a integridade e a segurança de transações financeiras.

### **4.3. Controle de Versão**

O controle de versão do código-fonte é gerenciado com o Git. O projeto é hospedado em um repositório remoto no GitHub, facilitando a colaboração, o rastreamento de alterações e a integração contínua.

### **4.4. Front-end**

A interface do usuário (UI) foi construída utilizando HTML e JavaScript, com a estilização baseada no framework Bootstrap. O uso do Bootstrap permitiu o desenvolvimento ágil de uma interface limpa, organizada e responsiva, garantindo uma boa experiência de uso sem a complexidade de bibliotecas reativas adicionais.

## 4.5. Infraestrutura

A infraestrutura do projeto opera de forma híbrida:

- Banco de Dados: Hospedado na nuvem através da plataforma Render, o que assegura a persistência dos dados e permite o acesso remoto.
- Aplicação (Back-end e Front-end): Executados em ambiente local. O servidor Flask roda localmente conectando-se ao banco na nuvem, enquanto o front-end consome a API localmente.

## 5. CRONOGRAMA DE ATIVIDADES

O desenvolvimento do projeto foi planejado para ser executado em um período de 10 semanas, dividido em quatro fases sequenciais. A estrutura foi desenhada para otimizar o tempo, permitindo a evolução escalonada do back-end para o front-end e garantindo que os testes ocorram de forma contínua.

### 5.1. Fase 1: Análise e Projeto (Semanas 1 a 2)

Foco na definição dos requisitos e na elaboração da arquitetura fundamental do sistema.

- Semana 1: Levantamento de Requisitos e Modelagem de Dados
  - Definição dos requisitos funcionais e não funcionais.
  - Criação do Modelo Entidade-Relacionamento (MER) e do projeto lógico do banco de dados.
- Semana 2: Projeto da Arquitetura de Software
  - Elaboração do diagrama de classes para o back-end.
  - Desenho da arquitetura da API REST, definindo endpoints e contratos de dados (JSON).

### 5.2. Fase 2: Desenvolvimento do Back-end (Semanas 3 a 5)

Implementação da lógica de negócio, da API e da camada de persistência.

- Semana 3: Configuração e Base do Projeto
  - Configuração do ambiente de desenvolvimento (Python, Flask, psycopg2).
  - Implementação das classes e scripts de conexão com o banco de dados PostgreSQL.
- Semana 4: Implementação dos Endpoints

- Desenvolvimento das rotas da API para as operações CRUD das entidades principais (Clientes, Contas).
- Semana 5: Lógica de Negócio e Segurança
  - Implementação das regras para transações financeiras (depósitos, saques, transferências).
  - Implementação dos mecanismos de validação de dados.

### **5.3. Fase 3: Desenvolvimento do Front-end e Integração (Semanas 6 a 8)**

Construção da interface do usuário e sua integração com os serviços do back-end.

- Semana 6: Design e Estruturação da Interface
  - Criação dos wireframes e da estrutura inicial das páginas HTML.
  - Estilização das telas principais utilizando o framework Bootstrap.
- Semana 7: Integração com a API
  - Desenvolvimento de scripts em JavaScript para conectar o front-end aos endpoints do back-end.
  - Manipulação do DOM para exibição dinâmica dos dados recebidos da API.
- Semana 8: Refinamento e Testes de Integração
  - Ajuste fino da responsividade e da experiência do usuário (UX) com componentes do Bootstrap.
  - Validação da comunicação entre o cliente local e o servidor remoto.

### **5.4. Fase 4: Testes Finais, Implantação e Documentação (Semanas 9 a 10)**

Garantia da qualidade, publicação da aplicação e conclusão da documentação.

- Semana 9: Testes de Sistema e Correções
  - Execução de testes manuais de ponta a ponta para validar os fluxos da aplicação.
  - Identificação e correção de bugs e tratamento de exceções.
- Semana 10: Implantação e Finalização
  - Publicação (deploy) do back-end na plataforma de nuvem Render.
  - Configuração final do ambiente local para execução do front-end.
  - Revisão e entrega da documentação técnica final do projeto.

## **6. DEFINIÇÕES DE REQUISITOS**

O levantamento e a análise de requisitos constituem a etapa fundamental para o desenvolvimento de qualquer sistema de software, pois definem o escopo, os objetivos e as restrições do projeto (Sommerville, 2017). Nesta fase, as necessidades dos stakeholders são traduzidas em especificações técnicas que guiarão as etapas subsequentes de modelagem e implementação do banco de dados.

Para este projeto, a análise se concentrou nas funcionalidades essenciais de uma aplicação de pagamentos digitais, simulando as operações de uma Fintech. O processo envolveu a decomposição do sistema em módulos gerenciáveis, resultando na identificação de Requisitos Funcionais (RF), que descrevem o comportamento e as operações que o sistema deve executar, e Requisitos Não Funcionais (RNF), que especificam critérios de qualidade, como desempenho, segurança e usabilidade (Pressman; Maxim, 2016). As seções a seguir detalham cada uma dessas categorias.

### **6.1. Requisitos Funcionais (RF)**

Os Requisitos Funcionais definem as funcionalidades específicas que o sistema deve prover aos seus usuários. Com base no diagrama entidade-relacionamento e no escopo do projeto, os seguintes requisitos foram estabelecidos:

RF01 – Gerenciamento de Clientes: O sistema deve prover funcionalidades para o ciclo de vida do cliente.

- RF01.1: Permitir o cadastro completo de um novo cliente, incluindo dados pessoais (nome, CPF, data de nascimento), de contato (endereço, telefone, e-mail) e profissionais.
- RF01.2: Permitir a autenticação segura do cliente por meio de credenciais (e.g., usuário e senha) para acesso à plataforma.

RF02 – Gestão de Contas: O sistema deve suportar a administração de contas financeiras vinculadas aos clientes.

- RF02.1: Permitir a criação de contas digitais associadas a um cliente, definindo seu tipo (e.g., Conta Corrente, Conta Poupança).
- RF02.2: Atribuir a cada conta um número e agência únicos, e registrar seu estado (e.g., ativa, bloqueada).
- RF02.3: Permitir que o cliente consulte o saldo e o extrato detalhado de sua conta em tempo real.

RF03 – Processamento de Transações Financeiras: O sistema deve ser capaz de executar operações financeiras de forma atômica e segura.

- RF03.1: Permitir a realização de depósitos em uma conta, atualizando seu saldo.
- RF03.2: Permitir a execução de saques, condicionados à verificação de saldo suficiente na conta de origem.
- RF03.3: Permitir a transferência de valores entre contas da própria instituição, validando o saldo da conta de origem antes de confirmar a operação.

RF04 – Auditoria e Histórico: O sistema deve manter um registro completo e imutável das operações.

- RF04.1: Registrar todas as movimentações financeiras (depósitos, saques, transferências) em um histórico de transações, incluindo data, hora, tipo, valor e contas envolvidas.
- RF04.2: Manter um log de auditoria (Log\_Auditoria) para registrar alterações em tabelas críticas do sistema, armazenando dados antigos e novos para fins de rastreabilidade.

## **6.2. Requisitos Não Funcionais (RNF)**

Os Requisitos Não Funcionais estabelecem os critérios de qualidade e as restrições operacionais do sistema. Eles são essenciais para garantir que a plataforma seja não apenas funcional, mas também eficiente, segura e confiável.

RNF01 – Usabilidade: A interação do usuário com o sistema, por meio de sua interface, deve ser intuitiva e eficiente. A arquitetura do banco de dados deve suportar consultas rápidas que permitam uma experiência de usuário fluida, minimizando tempos de carregamento.

RNF02 – Segurança: O sistema deve garantir a confidencialidade, integridade e disponibilidade dos dados.

- RNF02.1: Confidencialidade: Dados sensíveis, como senhas, devem ser armazenados de forma criptografada (e.g., utilizando algoritmos de hashing com salt). O sistema deve também suportar funcionalidades de privacidade na interface, como a ocultação de saldo.
- RNF02.2: Integridade: As senhas de usuário devem obedecer a uma política de complexidade, exigindo no mínimo 8 caracteres, com a inclusão de letras maiúsculas, minúsculas, números e caracteres especiais.
- RNF02.3: Controle de Acesso: O acesso aos dados deve ser restrito com base nas permissões do usuário autenticado.

RNF03 – Desempenho: O tempo de resposta para as operações críticas deve ser mínimo. As consultas para visualização de saldo e extrato devem ser executadas em menos de 2 segundos, mesmo com um grande volume de dados. As transações (PIX, TED) devem ser confirmadas em tempo real.

RNF04 – Confiabilidade: O sistema deve ser robusto e garantir a consistência dos dados financeiros. As transações devem seguir as propriedades ACID. O sistema deve ter uma disponibilidade de 99.9%, minimizando a possibilidade de falhas que impeçam o acesso dos usuários.

RNF05 – Escalabilidade: A arquitetura do banco de dados deve ser projetada para suportar um aumento no número de clientes, contas e transações sem degradação significativa do desempenho.

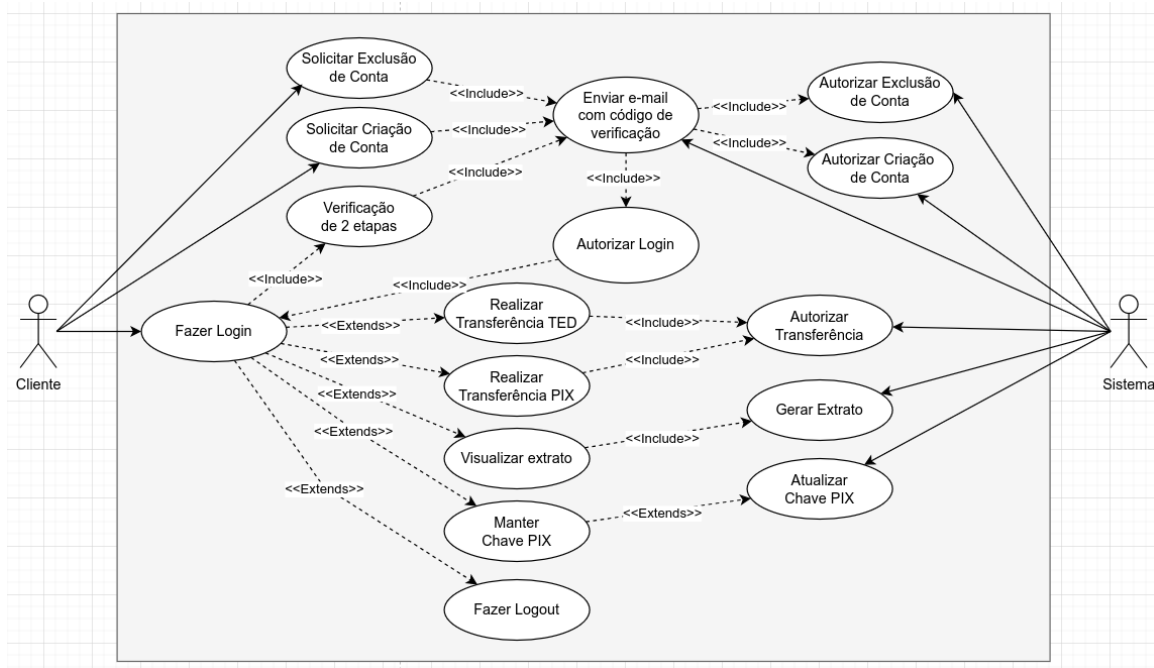
## **7. PROJETO DE SOFTWARE**

Esta seção detalha a arquitetura e o comportamento do sistema do ponto de vista do usuário e dos processos. Utilizaremos diagramas da UML (Unified Modeling Language) para ilustrar os Casos de Uso, que descrevem as interações entre os atores e o sistema, e os Diagramas de Atividades, que mapeiam os fluxos de trabalho do sistema.

### **7.1. Casos de Uso**

O diagrama de Casos de Uso a seguir, Figura 1, representa as funcionalidades do sistema e como os atores (Cliente e Sistema) interagem com elas.

**Figura 1 – Diagrama de Casos de Uso**



Fonte: Autores (2025).

A seguir, são detalhados os principais casos de uso identificados no diagrama:

#### UC01 - Fazer Login

- Ator: Cliente
- Descrição: Este caso de uso permite que um cliente já cadastrado acesse sua conta no sistema. O processo é um pré-requisito para a maioria das outras funcionalidades, como realizar transferências e consultar extratos.
- Relações:
  - <<Include>> Verificação de 2 etapas: Para garantir a segurança, o login sempre acionará um processo de verificação em duas etapas.
  - <<Extend>>: Após o login bem-sucedido, o cliente pode estender a sua sessão para: Realizar Transferência TED, Realizar Transferência PIX, Visualizar extrato, Manter Chave PIX e Fazer Logout.

#### UC02 - Solicitar Criação de Conta

- Ator: Cliente
- Descrição: Permite que um novo usuário solicite a criação de uma conta no sistema. O cliente fornecerá os dados necessários e aguardará a autorização.
- Relações:
  - <<Include>> Enviar e-mail com código de verificação: Como parte do processo, o sistema envia um e-mail de verificação para validar o endereço eletrônico do cliente.

#### UC03 - Solicitar Exclusão de Conta

- Ator: Cliente
- Descrição: Permite que um cliente existente solicite o encerramento de sua conta.
- Relações:
  - <<Include>> Enviar e-mail com código de verificação: O sistema envia um e-mail para confirmar a solicitação de exclusão, garantindo que a ação foi de fato iniciada pelo titular da conta.

#### UC04 - Realizar Transferência PIX / TED

- Ator: Cliente
- Descrição: Após o login, o cliente pode iniciar uma transferência de fundos para outra conta, escolhendo entre as modalidades PIX ou TED. O sistema validará os dados e solicitará autorização para completar a transação.
- Relações:
  - <<Include>> Autorizar Transferência: O sistema interno deve autorizar a transação antes que ela seja efetivada.

#### UC05 - Manter Chave PIX

- Ator: Cliente
- Descrição: Permite ao cliente gerenciar suas chaves PIX, incluindo a criação, alteração ou remoção de chaves associadas à sua conta.
- Relações:
  - <<Extend>> Atualizar Chave PIX: Ação executada pelo sistema para efetivar as mudanças solicitadas pelo cliente no banco de dados.

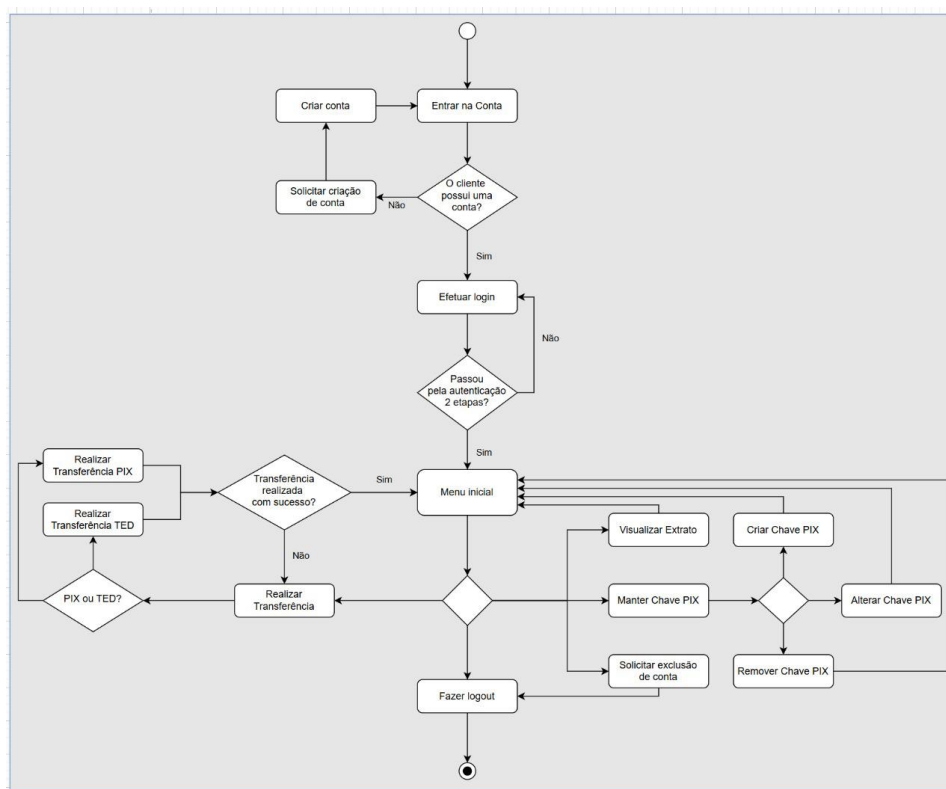
#### UC06 - Visualizar Extrato

- Ator: Cliente
- Descrição: O cliente pode consultar o extrato de sua conta, visualizando o histórico de transações (entradas e saídas) em um determinado período.
- Relações:
  - <<Include>> Gerar Extrato: O sistema é responsável por compilar as informações e gerar o extrato a ser apresentado.

## 7.2. Diagrama de Atividades

O diagrama de atividades a seguir, Figura 2, ilustra o fluxo de trabalho geral do sistema, desde o acesso inicial até a execução das principais funcionalidades. Ele detalha as decisões e os caminhos que um usuário pode seguir durante a utilização da aplicação.

**Figura 2 – Diagrama de Atividades**



Fonte: Autores (2025).

O fluxo de atividades do sistema pode ser descrito da seguinte forma:

1. Início: O processo começa quando o usuário acessa a tela inicial do sistema, onde pode optar por Entrar na Conta ou Criar conta.
2. Criação de Conta: Se o usuário não possui uma conta, ele pode selecionar a opção para Solicitar criação de conta. Após a solicitação, ele retorna à tela inicial.
3. Login: Caso o usuário já tenha uma conta, ele prossegue para Efetuar login, inserindo suas credenciais.
4. Autenticação de 2 Etapas: Por segurança, o sistema exige uma autenticação de 2 etapas. Se a autenticação falhar, o usuário retorna à tela de login. Se for bem-sucedida, ele avança para o Menu Inicial.
5. Menu Inicial: Esta é a área principal da aplicação, onde o usuário tem acesso a todas as funcionalidades:

6. Realizar Transferência: O usuário pode optar por fazer uma transferência. O sistema perguntará se a modalidade é PIX ou TED. Após a seleção e a execução (Realizar Transferência PIX ou Realizar Transferência TED), o sistema informa se a operação foi bem-sucedida e retorna ao menu.

7. Visualizar Extrato: O usuário pode acessar seu histórico de transações e, em seguida, retornar ao menu.

8. Manter Chave PIX: Nesta opção, o usuário pode gerenciar suas chaves, escolhendo entre Criar Chave PIX, Alterar Chave PIX ou Remover Chave PIX. Após a ação, ele volta ao menu.

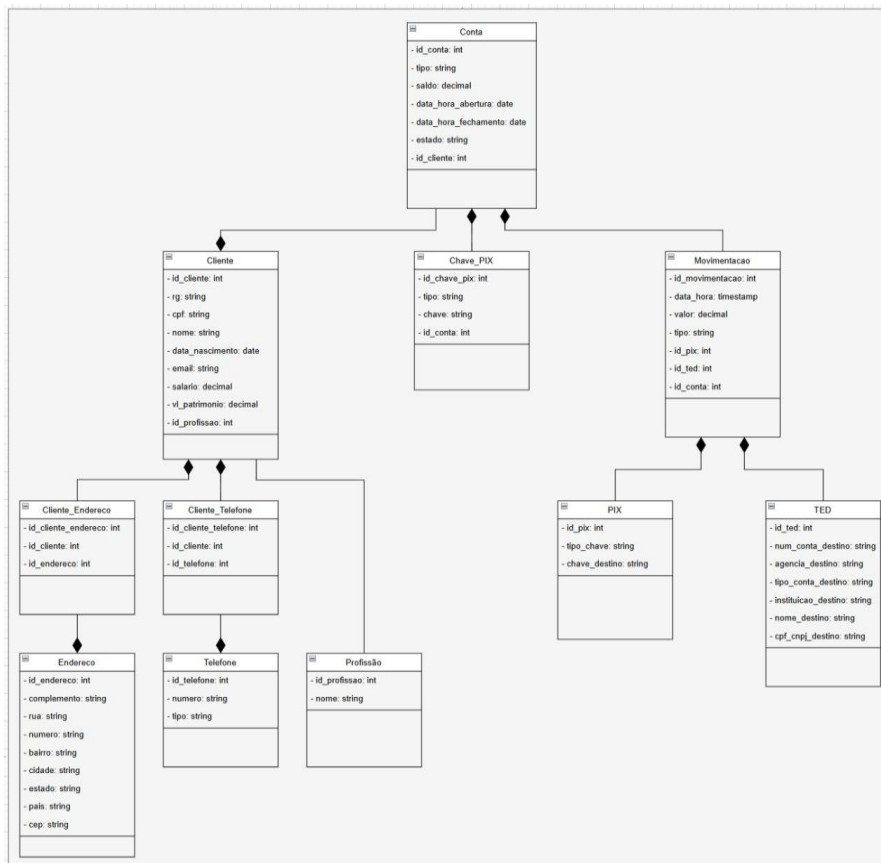
9. Solicitar exclusão de conta: O usuário pode iniciar o processo para encerrar sua conta e, depois, retorna ao menu.

10. Fazer logout: O usuário encerra sua sessão de forma segura, finalizando o processo.

### **7.3. Diagrama de Classes**

O diagrama de classes a seguir, Figura 3, modela a estrutura estática do sistema, definindo as entidades de negócio, seus atributos e os relacionamentos entre elas. Este diagrama serve como base para a construção do banco de dados e a organização do código-fonte.

**Figura 3 – Diagrama de Classes**



Fonte: Autores (2025).

#### Descrição das Classes:

- **Conta:** A classe central do sistema. Representa a conta bancária do cliente. Armazena informações como tipo de conta, saldo, datas de abertura/fechamento e estado (ativa, inativa, bloqueada).
- **Cliente:** Representa o titular da conta. Contém dados pessoais essenciais como nome, CPF, RG, data de nascimento, e-mail e informações patrimoniais e profissionais.
- **Endereco:** Armazena os dados de endereço do cliente, como rua, número, cidade, CEP, etc.
- **Telefone:** Armazena os números de telefone de contato do cliente, com um tipo (celular, residencial, etc.).
- **Profissao:** Contém a profissão do cliente.
- **Chave\_PIX:** Modela uma chave PIX associada a uma conta, contendo seu tipo (CPF, e-mail, celular, aleatória) e a chave em si.
- **Movimentacao:** Registra toda e qualquer transação financeira (crédito ou débito) ocorrida na conta, armazenando o valor e a data/hora.

- PIX: Especialização de uma Movimentacao, contendo os detalhes específicos de uma transação via PIX, como a chave de destino.

- TED: Especialização de uma Movimentacao, com os dados necessários para uma transferência via TED, como banco, agência, conta e dados do destinatário.

#### Descrição dos Relacionamentos

- Conta e Cliente: Existe uma relação de agregação forte (composição) onde uma Conta deve pertencer a um e somente um Cliente.

- Cliente, Endereco e Telefone: Um Cliente pode ter múltiplos endereços e telefones. As classes Cliente\_Endereco e Cliente\_Telefone atuam como tabelas associativas para materializar essa relação de "muitos-para-muitos".

- Conta, Chave\_PIX e Movimentacao: Uma Conta pode possuir várias Chave\_PIX e um histórico de múltiplas Movimentacao(s). A relação é de composição, pois chaves PIX e movimentações não existem sem uma conta associada.

- Movimentacao, PIX e TED: Uma Movimentacao está associada a uma transação específica, que pode ser do tipo PIX ou TED. O diagrama indica que cada registro de Movimentacao estará ligado a um registro detalhado em PIX ou TED, dependendo do tipo da transação.

## 8. BANCO DE DADOS

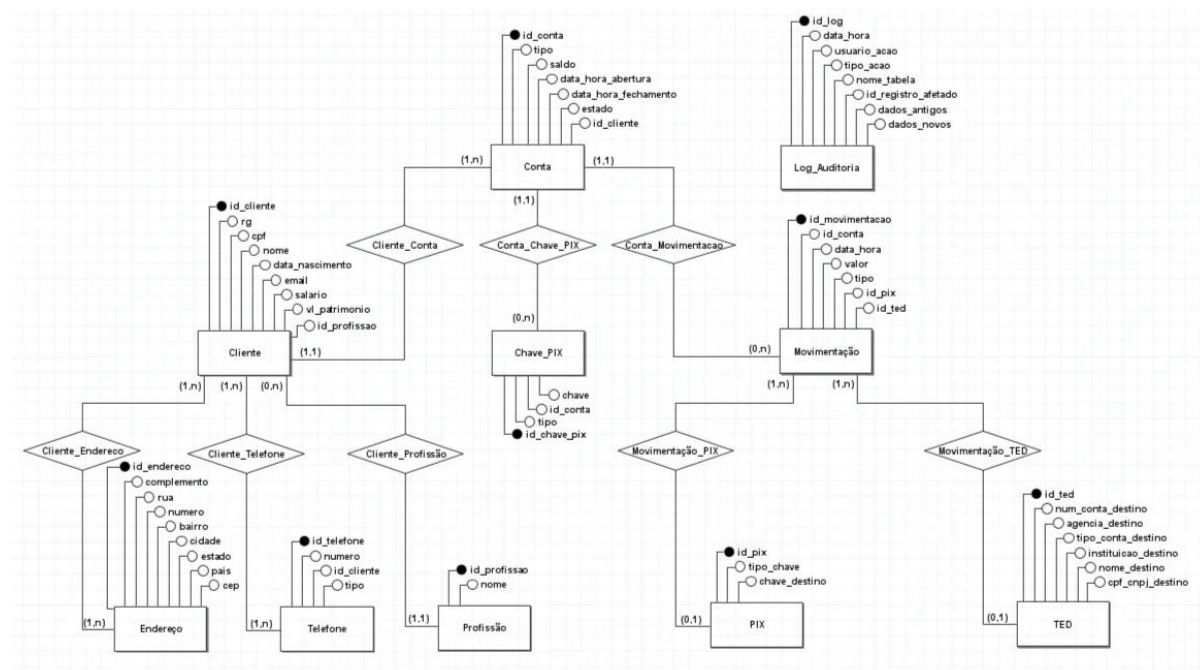
A arquitetura de dados constitui a espinha dorsal de qualquer aplicação robusta, sendo especialmente crítica em sistemas transacionais como uma plataforma de pagamentos digitais. É o banco de dados que garante a segurança, a integridade e a disponibilidade das informações vitais, desde os dados cadastrais dos usuários até o histórico detalhado de cada operação financeira. Uma falha ou ineficiência em sua estrutura pode comprometer não apenas a performance do sistema, mas também a confiança dos clientes e a conformidade regulatória do negócio.

Com o objetivo de construir uma base sólida e escalável, o projeto do nosso banco de dados foi desenvolvido seguindo uma abordagem metodológica em fases. Esta seção detalha essa jornada de modelagem, iniciando pelo projeto conceitual, que traduz as regras de negócio em um modelo abstrato. Em seguida, avançaremos para os projetos lógico e físico, onde a estrutura é refinada e otimizada para o Sistema Gerenciador de Banco de Dados (SGBD) escolhido, garantindo eficiência e consistência para a plataforma.

## 8.1. Projeto Conceitual

O projeto conceitual define a estrutura de alto nível do banco de dados, focando nas entidades essenciais e em como elas se relacionam para atender às regras de negócio. O Diagrama Entidade-Relacionamento (DER) apresentado, Figura 4, ilustra a arquitetura de dados para uma plataforma de serviços financeiros.

**Figura 4 - Diagrama Entidade-Relacionamento do Banco de Dados**



Fonte: Autores (2025).

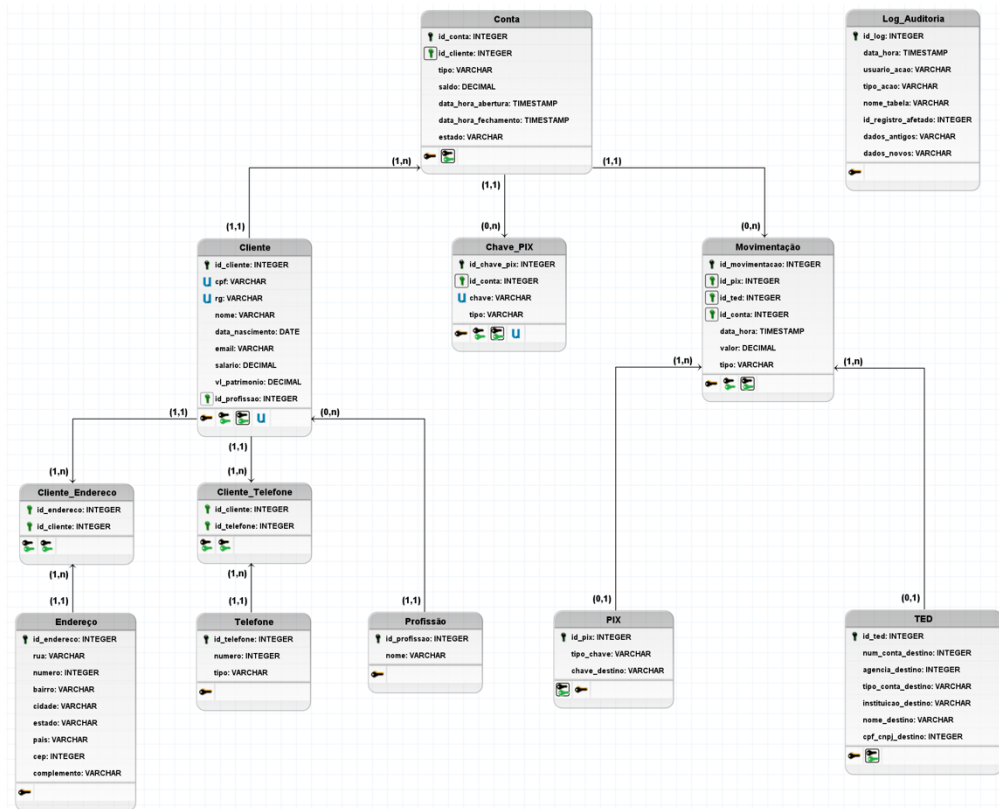
O modelo é centrado na entidade Cliente, que armazena os dados cadastrais e se conecta a informações complementares como Endereço, Telefone e Profissão. Cada cliente possui uma Conta para a gestão de seu saldo e operações. Todas as transações são registradas na entidade Movimentação, que se especializa em tipos específicos de operação, como PIX e TED. A entidade Chave\_PIX é associada diretamente à conta para viabilizar as transferências. Por fim, a entidade Log\_Auditoria garante a rastreabilidade e a segurança ao registrar todas as alterações relevantes no sistema.

## 8.2. Projeto Lógico

O projeto lógico é a fase intermediária da modelagem de banco de dados, na qual o modelo conceitual abstrato é traduzido em uma estrutura detalhada, compatível com um modelo de banco de dados específico, neste caso, o relacional. Nesta etapa, as entidades e relacionamentos são convertidos em tabelas, colunas, chaves primárias (PK), chaves estrangeiras (FK) e restrições de integridade. O objetivo é refinar o esquema, definindo os tipos

de dados e as regras que garantirão a consistência e a integridade dos dados, preparando o terreno para a implementação física.

**Figura 5 – Modelo Lógico do Banco de Dados**



Fonte: Autores (2025).

O diagrama acima, Figura 5, ilustra o modelo lógico da nossa plataforma. As entidades do projeto conceitual foram mapeadas para as seguintes tabelas:

- Tabelas Principais: Cliente, Conta, Movimentacao, Chave\_PIX, PIX, TED e Log\_Auditoria.
- Tabelas Auxiliares: Endereco, Telefone e Profissao.
- Tabelas de Ligação: Cliente\_Endereco e Cliente\_Telefone, que implementam a associação entre um cliente e seus múltiplos endereços e telefones.

Os relacionamentos são implementados através de chaves estrangeiras, que conectam as tabelas e garantem a integridade referencial. Por exemplo:

- A tabela Conta possui a coluna id\_cliente como chave estrangeira, estabelecendo a relação de que uma conta pertence a um único cliente.
- A tabela Movimentacao se conecta à Conta através da chave estrangeira id\_conta.

- A especialização da Movimentacao em PIX ou TED é modelada com as chaves estrangeiras id\_pix e id\_ted na tabela Movimentacao, onde apenas uma delas será preenchida por transação.

A definição precisa dos tipos de dados é crucial para a performance e a integridade. Foram escolhidos tipos como INTEGER para identificadores únicos (chaves primárias), VARCHAR para textos, DECIMAL para valores monetários (garantindo precisão), e TIMESTAMP ou DATE para registros de data e hora. Além disso, foram aplicadas restrições (constraints) importantes, como a de unicidade (UNIQUE) para os campos Cliente.cpf e Chave\_PIX.chave, impedindo a duplicidade de registros e assegurando que as regras de negócio sejam aplicadas diretamente no banco de dados.

### **8.3. Projeto Físico**

A implementação do banco de dados foi realizada no SGBD PostgreSQL, hospedado na plataforma de nuvem Render. O esquema físico foi desenhado para garantir integridade referencial, consistência transacional e auditoria de dados. Abaixo, figura 6, é possível visualizar todos os objetos criados no projeto físico.

**Figura 6 – Objetos do projeto físico**

- sistema\_bancario\_db\_b6lw/admin@Projeto-DWEB

Query Query History

```

5 SELECT tipo_objeto, nome_objeto
6 FROM (
7     -- 1. TABELAS, VIEWS
8     SELECT
9         CASE c.relkind
10             WHEN 'r' THEN '1. TABELA'

```

Data Output Messages Notifications

Showing rows: 1 to 14

|    | tipo_objeto<br>text | nome_objeto<br>name               |
|----|---------------------|-----------------------------------|
| 1  | 1. TABELA           | chaves_pix                        |
| 2  | 1. TABELA           | clientes                          |
| 3  | 1. TABELA           | contas                            |
| 4  | 1. TABELA           | enderecos                         |
| 5  | 1. TABELA           | log_auditoria                     |
| 6  | 1. TABELA           | movimentacoes                     |
| 7  | 1. TABELA           | profissoes                        |
| 8  | 1. TABELA           | telefones                         |
| 9  | 2. VIEW             | view_clientes_gerencia            |
| 10 | 2. VIEW             | view_extrato_cliente              |
| 11 | 5. FUNÇÃO/PROCEDURE | auditar_alteracao_cliente         |
| 12 | 5. FUNÇÃO/PROCEDURE | realizar_saque                    |
| 13 | 5. FUNÇÃO/PROCEDURE | realizar_transferencia_interna    |
| 14 | 6. TRIGGER          | tr_auditar_clientes (em clientes) |

### 8.3.2. Lógica de Negócio no Banco (Stored Procedures)

Para assegurar a atomicidade das operações financeiras e evitar condições de corrida (race conditions), as transações críticas foram encapsuladas em procedimentos armazenados PL/pgSQL:

1. `realizar_saque(id_conta, valor)`: Verifica a existência da conta e a disponibilidade de saldo antes de efetivar o débito e registrar o histórico.
2. `realizar_transferencia_interna(origem, destino, valor)`:
  - Executa uma transação completa (débito na origem e crédito no destino) de forma atômica.
  - Utiliza a cláusula `FOR UPDATE` nos `selects` de validação, bloqueando temporariamente os registros das contas envolvidas para impedir que operações concorrentes alterem o saldo simultaneamente durante a transferência.

### 8.3.3. Auditoria e Segurança (Triggers)

Foi implementado um mecanismo de rastreabilidade para dados sensíveis dos clientes (Tabela `clientes`):

- Tabela `log_auditoria`: Armazena o histórico de alterações utilizando campos do tipo `JSONB` (`dados_antigos` e `dados_novos`). Isso permite guardar o estado completo do registro ("foto" dos dados) de forma flexível.
- Trigger `tr_auditar_clientes`: Gatilho automático disparado após eventos de `INSERT`, `UPDATE` ou `DELETE`, que invoca a função `auditar_alteracao_cliente()` para registrar quem alterou o dado, quando e o que foi modificado.

### 8.3.4. Camada de Apresentação (Views)

Foram criadas visões para simplificar as consultas realizadas pela aplicação e gerar relatórios:

- `view_extrato_cliente`: Abstrai a complexidade da tabela de movimentações, formatando as descrições das transações (ex: concatenando "PIX ENVIADO" com o nome do destino) e classificando o fluxo financeiro em "ENTRADA" ou "SAÍDA".
- `view_clientes_gerencial`: Realiza a junção (Join) das tabelas de clientes, contas, profissões e endereços, fornecendo uma visão consolidada para o perfil do correntista.

## 9. PROPOSTA COMERCIAL

Esta proposta detalha o investimento necessário para o desenvolvimento, implantação e operação inicial do sistema bancário da IFSP - Instituição Financeira de São Paulo. A estratégia comercial adotada prioriza o uso de tecnologias de código aberto e infraestrutura em nuvem escalável, visando reduzir o Custo Total de Propriedade (TCO) e acelerar o Time-to-Market do MVP (Minimum Viable Product).

### 9.1. Investimento em Desenvolvimento (CAPEX)

O custo de desenvolvimento foi calculado com base na estimativa de horas de engenharia necessárias para cobrir o ciclo de vida do projeto, desde a modelagem do banco de dados até a entrega final. A escolha da stack tecnológica (Python/Flask e Bootstrap) permitiu uma redução estimada de 20% no tempo de desenvolvimento comparado a arquiteturas mais complexas, resultando em maior eficiência orçamentária.

**Tabela 1 – Custo CAPEX**

| Fase do Projeto | Atividades Chave                                 | Horas Estimadas | Custo Estimado (R\$)* |
|-----------------|--------------------------------------------------|-----------------|-----------------------|
| 1. Planejamento | Levantamento de requisitos, MER e Arquitetura    | 40h             | R\$ 2.400,00          |
| 2. Back-end     | API REST, Lógica em Python e Integração psycopg2 | 80h             | R\$ 4.800,00          |
| 3. Front-end    | Interface Web Responsiva e Integração Local      | 60h             | R\$ 3.600,00          |
| 4. QA e Deploy  | Testes, Correções e Publicação na Nuvem          | 20h             | R\$ 1.200,00          |
| TOTAL           | Desenvolvimento do MVP                           | 200h            | R\$ 12.000,00         |

Fonte: Autores (2025).

\* Base de cálculo: Valor hora técnica de R\$ 60,00 (Referência de mercado para Desenvolvedor Full Stack Júnior).

### 9.2. Custos Operacionais e Infraestrutura (OPEX)

Para garantir a sustentabilidade financeira da operação nos primeiros meses, o projeto utiliza a modalidade PaaS (Platform as a Service) da Render, que elimina custos iniciais com servidores físicos e manutenção de hardware.

- Infraestrutura de Servidor (Back-end):
  - Estágio Atual: Plano Free Tier (Gratuito) – Suporta o volume de transações de teste e validação acadêmica.
  - Estágio de Produção: Plano Starter (Estimado em US\$ 7,00/mês) – Necessário para garantir uptime e performance com o aumento de usuários.
- Banco de Dados (PostgreSQL):
  - Hospedado como serviço gerenciado, com backups automáticos e alta disponibilidade, iniciando no plano gratuito durante a fase de homologação.
- Licenciamento de Software:
  - Custo Zero (R\$ 0,00): Todas as tecnologias empregadas (Python, PostgreSQL, Flask, Bootstrap) operam sob licenças open source (MIT, BSD ou similar), isentando o projeto de mensalidades ou royalties.

### **9.3. Retorno sobre o Investimento (ROI)**

A solução proposta oferece um excelente custo-benefício. Ao manter o front-end desacoplado e rodando localmente nesta fase, reduzimos o consumo de banda e processamento no servidor, otimizando os recursos da nuvem. O sistema está pronto para escalar conforme a demanda da IFSP, permitindo que o investimento em infraestrutura cresça apenas na medida em que a base de clientes aumentar.

## **10. CONSIDERAÇÕES FINAIS**

O desenvolvimento do Sistema Bancário permitiu a aplicação prática de conceitos fundamentais da Engenharia de Software e Banco de Dados, consolidando o conhecimento na construção de aplicações distribuídas. O objetivo principal — criar uma simulação funcional de uma instituição financeira capaz de realizar operações críticas como cadastro de contas e transações financeiras — foi atingido com sucesso.

A escolha da arquitetura, separando o back-end (hospedado no Render) do front-end (executado localmente), proporcionou um cenário realista de desenvolvimento de APIs RESTful. A utilização da biblioteca `psycpg2` ofereceu controle granular sobre as interações com o banco de dados PostgreSQL, permitindo a implementação otimizada das regras de negócio e garantindo a integridade transacional necessária para um sistema financeiro.

No front-end, o uso do Bootstrap possibilitou a criação ágil de uma interface amigável e responsiva, sem a complexidade adicional de frameworks reativos robustos, adequando-se perfeitamente ao escopo e prazo do projeto.

Embora o sistema atenda aos requisitos funcionais propostos, algumas melhorias podem ser implementadas em iterações futuras:

- **Segurança:** Implementação de autenticação via tokens (JWT) para proteger os endpoints da API.
- **Arquitetura Front-end:** Migração da interface local para uma hospedagem em nuvem e, eventualmente, adoção de um framework como React ou Vue.js para melhor gerenciamento de estado.
- **Relatórios:** Desenvolvimento de um módulo administrativo para geração de relatórios gerenciais e auditoria de transações.

Em suma, o projeto demonstrou a viabilidade técnica da solução proposta, entregando um produto de software funcional, documentado e pronto para evoluir.

## **11. REFERÊNCIAS BIBLIOGRÁFICAS**

LUCIANO, Francisco Veríssimo. SPOBDD2 – Banco de Dados 2 - Notas de aulas disponíveis no Ambiente Virtual de Aprendizagem Moodle do IFSP – Campus São Paulo. São Paulo: 2025.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 9. ed. Porto Alegre: AMGH, 2021.

PUSCHMANN, Thomas. Fintech. Business & Information Systems Engineering, [S. l.], v. 59, n. 1, p. 69-76, fev. 2017.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson Education do Brasil, 2017.